Greedy in Approximation Algorithms*

Julián Mestre

Department of Computer Science
University of Maryland, College Park, MD 20742

Abstract. The objective of this paper is to characterize classes of problems for which a greedy algorithm finds solutions provably close to optimum. To that end, we introduce the notion of k-extendible systems, a natural generalization of matroids, and show that a greedy algorithm is a $\frac{1}{k}$-factor approximation for these systems. Many seemly unrelated problems fit in our framework, e.g.: b-matching, maximum profit scheduling and maximum asymmetric TSP.

In the second half of the paper we focus on the maximum weight b-matching problem. The problem forms a 2-extendible system, so greedy gives us a $\frac{1}{2}$-factor solution which runs in $O(m \log n)$ time. We improve this by providing two linear time approximation algorithms for the problem: a $\frac{1}{2}$-factor algorithm that runs in $O(bm)$ time, and a $\left(\frac{2}{3} - \epsilon\right)$-factor algorithm which runs in expected $O\left(bm \log \frac{1}{\epsilon}\right)$ time.

1 Introduction

Perhaps the most natural first attempt at solving any combinatorial optimization problem is to design a greedy algorithm. The underlying idea is simple: we make locally optimal choices hoping that this will lead us to a globally optimal solution. Needless to say that such an algorithm may not always work, therefore a natural question to ask is: for which class of problems does this approach work? A classical theorem due to Edmonds and Rado answers this question; to state this result we first need to define our problem more rigorously.

A *subset system* is a pair $(E, \mathcal{L})$, where E is a finite set of elements and $\mathcal{L}$ is a collection of subsets of E such that if $A \in \mathcal{L}$ and $A' \subseteq A$ then $A' \in \mathcal{L}$. Sets in $\mathcal{L}$ are called *independent*, and should be regarded as feasible solutions of our problem. Given a positive weight function $w : E \to \mathbb{R}^+$ there is a natural optimization problem associated with $(E, \mathcal{L})$ and w, namely that of finding an independent set of maximum weight. We want to study the following algorithm, which from now on we simply refer to as Greedy: start from the empty solution and process the elements in decreasing weight order, add an element to the current solution only if its addition preserves independence.

A matroid is a subset system $(E, \mathcal{L})$ for which the following property holds:

$$\forall\, A, B \in \mathcal{L} \text{ and } |A| < |B| \text{ then } \exists\, z \in B \setminus A \text{ such that } A + z^1 \in \mathcal{L}$$

* Research supported by NSF Awards CCR-01-05413 and CCF-04-30650, and the University of Maryland Dean's Dissertation Fellowship.

[1] The notation $A + z$ means $A \cup \{z\}$, likewise $A - z$ means $A \setminus \{z\}$.

Y. Azar and T. Erlebach (Eds.): ESA 2006, LNCS 4168, pp. 528–539, 2006.
© Springer-Verlag Berlin Heidelberg 2006

Matroids were first introduced by Whitney [24] as an abstraction of the notion of independence from linear algebra and graph theory. Rado [21] showed that if a given problem has the matroid property then Greedy always finds an optimal solution. In turn, Edmonds [11] proved the other direction of the implication, i.e., if Greedy finds an optimal solution for *any* weight function defined on the elements then the problem must have the matroid property.

A rich theory of matroids exists, see [22, 18] for a thorough treatment of the subject. Many generalizations along two main directions have been proposed. One approach is to define a more general class of problems. Greedy no longer works, therefore alternative algorithms must be designed; examples of this are greedoids [15], two-matroid intersection [10], and matroid matching [17]. Another approach is to study structures where Greedy finds optimal solutions for some, but not all weight functions; symmetric matroids [7], sympletic matroids [6] and the work of Vince [23] are along these lines.

Although different in nature, both approaches have the same objective in mind: exact solutions. In this paper we study Greedy from the point of view of approximation algorithms. Our main contribution is the introduction of k-extendible systems, a natural generalization of matroids. We show that Greedy is a $\frac{1}{k}$-factor approximation for k-extendible systems.

Given a subset system $(E, \mathcal{L})$, Korte and Hausman [14] showed that for the maximization problem defined by $(E, \mathcal{L})$, Greedy achieves its worst approximation ratio on 0-1 weight functions. Consider the 0-1 function w_A defined as $w_A(x) = 1$ for $x \in A$ and 0 otherwise. The cost of the solution Greedy finds, comes from the elements in A the algorithm happens to pick, these elements form an independent set which is maximal with respect to A. Let γ_A be the ratio between the smallest and the largest maximal independent subsets of A. Notice that γ_A is the worst greedy can do on w_A. Let $\gamma = \min_{A \subseteq E} \gamma_A$. Korte and Hausman showed that Greedy is a γ-factor approximation for $(E, \mathcal{L})$.

While this result tells us how well Greedy performs on a particular system, in some cases it may be difficult to establish γ for a given combinatorial problem—which can be regarded as a class of systems, as every instance of the problem defines a system. Our k-extendible framework better highlights the structure of the problem and allows us to easily explain the performance of Greedy on seemingly unrelated problems such as b-matching, maximum profit scheduling and maximum asymmetric TSP. For some of these, an algorithm tailored to the specific problem yields a better approximation ratio than that offered by Greedy. This should not come as a surprise, after all Greedy is a generic algorithm that we can try on nearly every problem. The goal of this paper is to characterize those problems for which a simple greedy strategy produces nearly optimal solutions and to better understand its shortcomings. Along these lines is the recent work by Borodin et al. [5], who introduced the paradigm of priority algorithms, a formal class of algorithms that captures most greedy-like algorithms. Lower bounds on the approximation ratio any priority algorithm can achieve were derived for scheduling [5], set cover, and facility location problems [1].